

Snowflake COPY Command, Error Management/Handling, and Task Scheduling/Automation

1 COPY COMMAND – ERROR HANDLING OPTIONS

When loading data using COPY INTO, Snowflake may encounter **bad records** (wrong data type, missing columns, invalid format, etc.).

Snowflake provides **error handling options** to control what should happen.

◆ Error Options

Option	Meaning
ABORT_STATEMENT (default)	Stop the entire load if any error occurs
CONTINUE	Load valid rows, skip bad rows
SKIP_FILE	Skip the entire file if any error occurs in that file

✓ Example

```
COPY INTO employee
FROM @my_stage/emp.csv
FILE_FORMAT = (TYPE = CSV)
ON_ERROR = 'CONTINUE';
```

Explanation

- Valid rows get loaded
- Invalid rows are skipped
- No error stops the load
- Bad rows are **not loaded**, but they can be captured using **validation mode**

2 VALIDATION MODE – FIND BAD ROWS BEFORE LOADING

Validation mode allows you to **preview errors** without loading data.

◆ Validation Mode Options

Option	Purpose
RETURN_ERRORS	Returns first error per file
RETURN_ALL_ERRORS	Returns all error rows
RETURN_ROWS	Returns rows that would be loaded

✅ Example – Find All Bad Rows

```
COPY INTO employee
FROM @my_stage/emp.csv
FILE_FORMAT = (TYPE = CSV)
VALIDATION_MODE = 'RETURN_ALL_ERRORS';
```

Explanation

- Data is **not loaded**
- Snowflake returns all rejected rows
- Used mainly for **debugging & error logging**

3 RESULT_SCAN – CAPTURE BAD ROWS INTO ERROR TABLE

RESULT_SCAN is a **system-defined table function** used to read query results using **Query ID**.

♦ Why RESULT_SCAN?

- Validation mode returns data
 - But returned data is **temporary**
 - We use RESULT_SCAN to **store rejected rows permanently**
-

Step-by-Step Process

Step 1 Run COPY with validation mode

```
COPY INTO employee
FROM @my_stage/emp.csv
FILE_FORMAT = (TYPE = CSV)
VALIDATION_MODE = 'RETURN_ALL_ERRORS';
```

Step 2 Get Query ID

```
SELECT LAST_QUERY_ID(-1);
```

Step 3 Insert into Error Table

```
INSERT INTO employee_error_log
SELECT *
FROM TABLE(RESULT_SCAN('01b8d2f3-0001-...'));
```

Important Points

- RESULT_SCAN data exists **only for 24 hours**
- Once inserted, error table data stays **permanently**
- Very common **interview question**

COPY HISTORY – TRACK LOADED FILES

Snowflake keeps metadata of loaded files to avoid duplicate loads.

◆ INFORMATION_SCHEMA.COPY_HISTORY

SELECT *

FROM TABLE(

INFORMATION_SCHEMA.COPY_HISTORY(

TABLE_NAME => 'EMPLOYEE',

START_TIME => DATEADD(DAY, -14, CURRENT_TIMESTAMP)

)

);

Key Points

- Shows **last 14 days**
- Includes:
 - Fully loaded files
 - Partially loaded files
 - Failed files
- Truncating the table **resets history**

◆ ACCOUNT_USAGE.COPY_HISTORY

SELECT *

FROM SNOWFLAKE.ACCOUNT_USAGE.COPY_HISTORY;

Feature	Value
Retention	365 days
Delay	45–90 minutes
Scope	Entire account

5 TASKS – AUTOMATION & SCHEDULING

Tasks allow you to **schedule SQL statements automatically**.

◆ Task Characteristics

- Executes **one SQL statement** OR **one stored procedure**
- Initially created in **SUSPENDED** state
- Requires **RESUME** to start

✅ Example – Create Task

```
CREATE OR REPLACE TASK employee_task
```

```
WAREHOUSE = my_wh
```

```
SCHEDULE = '1 MINUTE'
```

```
AS
```

```
COPY INTO employee
```

```
FROM @my_stage/emp.csv
```

```
FILE_FORMAT = (TYPE = CSV)
```

```
FORCE = TRUE;
```

Explanation

- Runs every minute
- FORCE = TRUE reloads files (used only for testing)
- In real projects → hourly / daily schedule

▶ Resume Task

```
ALTER TASK employee_task RESUME;
```

II Suspend Task (VERY IMPORTANT)

```
ALTER TASK employee_task SUSPEND;
```

⚠ Interview Tip: **Tasks consume credits if running**

6 MONITOR TASK EXECUTION

◆ SHOW TASKS

SHOW TASKS;

◆ TASK HISTORY

```
SELECT *  
  
FROM TABLE(  
  
  INFORMATION_SCHEMA.TASK_HISTORY(  
    TASK_NAME => 'EMPLOYEE_TASK'  
  )  
);
```

Similar to:

- QUERY_HISTORY
- COPY_HISTORY

7 CRON EXPRESSIONS – FLEXIBLE SCHEDULING

Snowflake supports **CRON expressions** for advanced scheduling.

◆ CRON Format (5 Fields)

MIN HOUR DAY_OF_MONTH MONTH DAY_OF_WEEK

✓ Example – Monday to Friday at 2 AM IST

SCHEDULE = 'USING CRON 0 2 * * 1-5 Asia/Kolkata'

Explanation

- 0 → minute
- 2 → hour
- * → every day
- * → every month
- 1-5 → Monday to Friday
- Timezone overrides account timezone

8 TIME ZONES IN SNOWFLAKE

◆ Default Time Zone

America/Los_Angeles (UTC -8)

◆ Change Account Time Zone

```
ALTER ACCOUNT SET TIMEZONE = 'Asia/Kolkata';
```

Result

```
SELECT CURRENT_TIMESTAMP;
```

- Shows **IST time**
- Used for reporting & task scheduling

9 USER-MANAGED vs SERVERLESS TASKS

◆ User-Managed Task

WAREHOUSE = my_wh

Feature	Value
Uses your warehouse	✓
You manage compute	✓
Most common	✓

◆ Serverless (Snowflake-Managed) Task

```
CREATE TASK task_name
```

```
USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'
```

Feature	Value
Snowflake manages compute	✓
No warehouse needed	✓
Slightly higher cost	⚠

Important Behavior

- If one run is still executing → next scheduled run is **skipped**
- Tasks can still be **suspended/resumed**

10 TASK TREE (PARENT-CHILD TASKS)

To run **multiple SQL statements**, create a **task tree**.

◆ Parent Task

```
CREATE TASK parent_task  
WAREHOUSE = my_wh  
SCHEDULE = '1 HOUR'  
AS  
CALL parent_proc();
```

◆ Child Task

```
CREATE TASK child_task  
AFTER parent_task  
AS  
CALL child_proc();
```

🔗 INTERVIEW QUICK POINTS

- Tasks start in **SUSPENDED**
- RESULT_SCAN retains data for **24 hours**
- COPY_HISTORY resets on table truncate
- Cron supports **time zones**
- Tasks run **one SQL or one procedure**
- Suspend tasks to avoid **credit loss**